ARCHITECTURE MOBILE

Android

Composants & Communication

Activity . Intents · Services · BroadcastReceivers
ContentProviders · Permissions

Dr N.BELGUERCHE

Activité — Définition

Une Activité fournit une interface visuelle pour l'interaction avec l'utilisateur. Typiquement, elle supporte une seule opération qu'un utilisateur puisse faire à la fois.

Principe fondamental

Chaque écran de l'application est représenté par une Activité. Une app Android est composée d'un ensemble d'Activités, chacune étant indépendante.

L'utilisateur interagit avec une seule Activité à la fois (celle au premier plan).

Exemples d'usage

- Voir un message email
- Afficher l'écran de login
- Rédiger un nouveau message
- Parcourir une liste de contacts
- Afficher une carte GPS
- Visionner une vidéo

Une application = plusieurs Activités indépendantes

Application de messagerie électronique

Tâches (Tasks) & Back Stack

01 ACTIVITÉ

Une tâche est un ensemble d'activités reliées avec lesquelles l'utilisateur interagit. Elles ne sont pas nécessairement fournies par une seule application.

Activity Back Stack

Android gère une pile de retour d'activité (LIFO — Last In, First Out) :

- Lancer une activité → empilée au sommet
- Presser le bouton Retour → dépile l'activité courante
- La pile peut contenir des activités de plusieurs apps

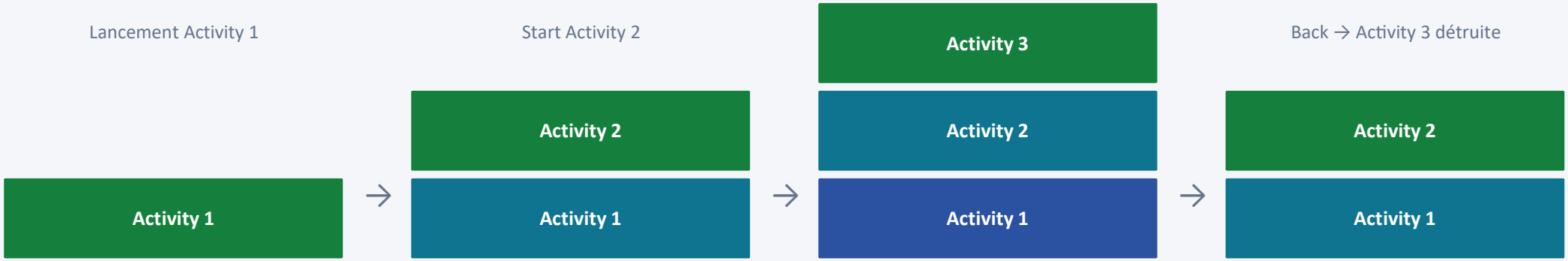
L'utilisateur peut remonter dans l'historique des écrans.

Comportement multi-applications

Exemple :
1. Gmail → sélectionne une pièce jointe PDF
2. Drive s'ouvre (nouvelle activité)
3. Bouton Retour → retour à Gmail

Les deux apps partagent la même tâche (back stack). L'utilisateur perçoit une expérience fluide et cohérente.

Visualisation du Back Stack



États d'une Activité

02 ÉTATS

Une Activity passe par différents états au cours de sa vie. Android peut tuer les activités en état Paused ou Stopped si la mémoire est insuffisante.

1 **Non lancée**

L'activité n'a pas encore été créée. Aucune instance n'existe en mémoire.

Risque de terminaison par Android →

2 **Active / Running**

L'activité est visible à l'écran et a le focus. L'utilisateur interagit avec elle. C'est l'état normal d'utilisation.

Jamais tuée

3 **Paused**

L'activité est visible mais n'a plus le focus (ex: une boîte de dialogue est au premier plan). Peut être tuée si la mémoire manque.

Rarement tuée

4 **Stopped**

L'activité n'est plus visible (une autre activité la couvre entièrement). L'état est préservé. Peut être tuée à tout moment.

Peut être tuée

5 **Terminée**

L'activité a été détruite (finish() appelé ou tuée par Android). Les ressources sont libérées.

Détruite

Cycle de Vie — Méthodes Callbacks

Android communique les changements d'état en appelant des méthodes spécifiques (callbacks) du cycle de vie. L'application surcharge ces méthodes pour réagir aux transitions.

`onCreate()`

Appelée une seule fois à la création. Initialise l'UI, lie les données. Obligatoire.

`onStart()`

L'activité devient visible. Préparer les ressources nécessaires à l'affichage.

`onResume()`

L'activité est au premier plan et reçoit les interactions. Démarrer les comportements foreground-only.

`onPause()`

L'activité perd le focus. Arrêter les comportements foreground-only, sauvegarder légèrement.

`onRestart()`

Appelée si l'activité était Stopped et redémarre (ex: retour du bouton Retour).

`onStop()`

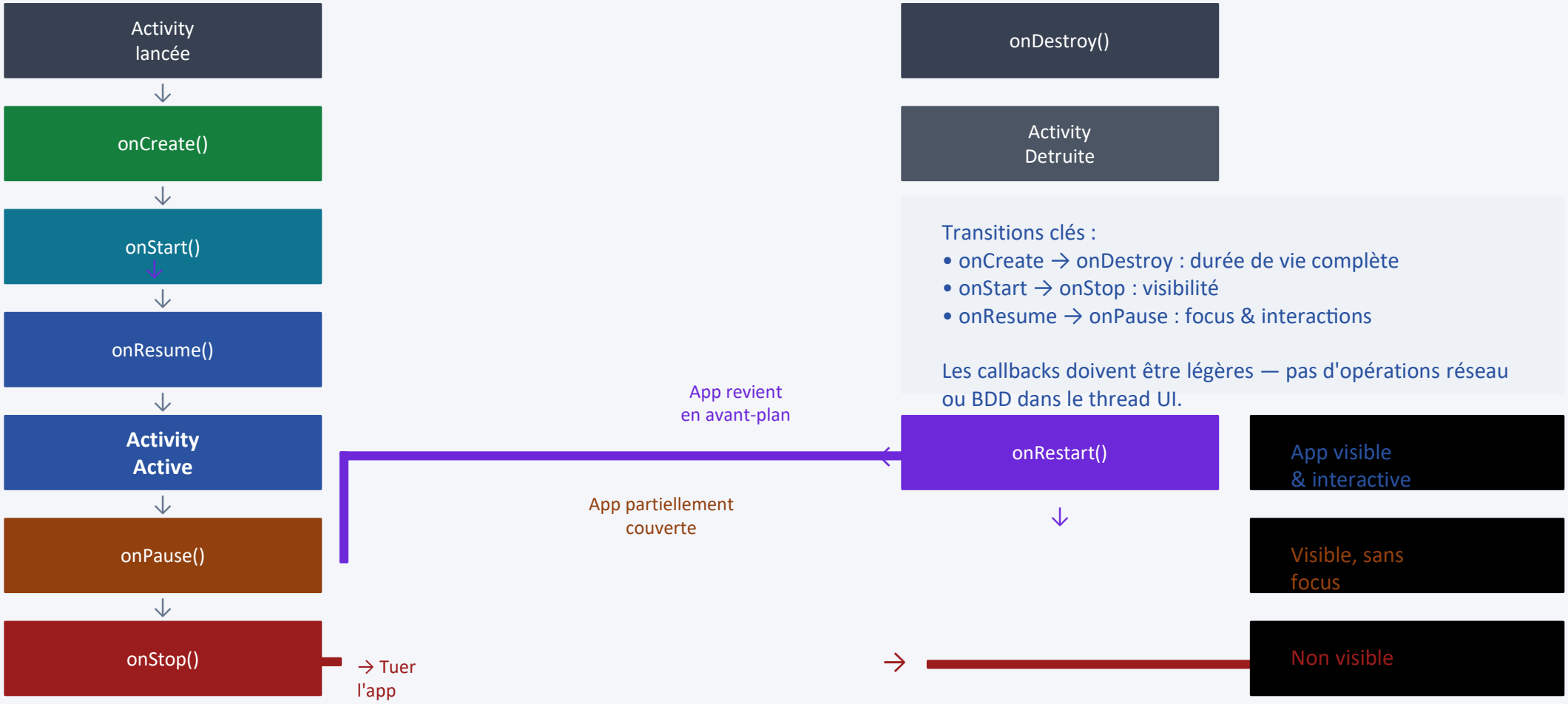
L'activité n'est plus visible. Libérer les ressources coûteuses. Sauvegarder l'état.

`onDestroy()`

Activité détruite. Libérer toutes les ressources. Dernière chance de nettoyage.

Important : toutes les callbacks s'exécutent dans le thread principal (UI thread). Ne jamais faire d'opérations longues ici — utiliser des coroutines ou AsyncTask.

Cycle de Vie — Diagramme Complet



Transitions clés :
- onCreate → onDestroy : durée de vie complète
- onStart → onStop : visibilité
- onResume → onPause : focus & interactions

Les callbacks doivent être légères — pas d'opérations réseau ou BDD dans le thread UI.

Sauvegarder l'État d'Activité

Les apps peuvent être tuées et redémarrées fréquemment (rotation d'écran, manque de mémoire). Il faut sauvegarder l'état pour restaurer une expérience cohérente.

onSaveInstanceState()

Appelée avant la destruction de l'activité.

- Sauvegarder l'état interne dans un Bundle
- Appelée lors de la rotation de l'appareil
- Appelée si Android tue l'app
- NON appelée si l'utilisateur ferme volontairement

Exemple : position de scroll, texte saisi, onglet actif

onRestoreInstanceState()

Appelée lors du redémarrage de l'activité si un Bundle est disponible.

- Restaurer l'état sauvegardé
- Appelée après onStart()
- Alternativement : lire savedInstanceState dans onCreate()

Exemple : repositionner le scroll, remplir les champs de formulaire

Cas déclencheurs

Rotation d'écran

Android détruit et recrée l'Activity pour charger les nouvelles ressources (layout landscape).

Pression mémoire

Android tue les apps en arrière-plan. Lors de la reprise, l'état doit être restauré.

Navigation système

Bouton Home → app en arrière-plan. Si tuée, l'état doit être restauré au retour.

Comment les Activités communiquent-elles ?

Une Activity seule ne suffit pas — elle a besoin d'Intents pour interagir avec le reste du système

Activity

Vu précédemment :
- Un écran = une Activity
- Cycle de vie géré par Android
- Back Stack (LIFO)
- Sauvegarde d'état
- Priorité processus

Mais comment passe-t-on d'une Activity à une autre ?

→

Intent

Le messager :
- Explicite : cible nommée
- Implicite : action décrite
- Porte des Extras (données)
- Route via Android OS

L'Intent est le seul moyen de démarrer Activity, Service ou Receiver

→

Composants cibles

À venir dans ce cours :
- **Activity (via Intent explicite)**
- **Service (startService / bindService)**
- **BroadcastReceiver (sendBroadcast)**
- **ContentProvider (ContentResolver)**

Qu'est-ce qu'un Intent ?

Un Intent (« Intention ») est un objet de messagerie Android. Il permet à un composant (Activity, Service...) de demander une action à un autre composant, qu'il soit dans la même application ou dans une autre entièrement différente.

Deux rôles fondamentaux

▶ Déclencher une action

L'Intent demande au système de réaliser quelque chose de précis :

- → Ouvrir une page web dans le navigateur
- → Composer un numéro de téléphone
- → Prendre une photo avec la caméra
- → Partager un texte via WhatsApp, Gmail...
- → Afficher un lieu sur une carte

📢 Signaler un événement système

L'Intent informe les applications qu'un événement s'est produit :

- → La batterie est passée sous 15 %
- → Le téléphone vient de démarrer
- → Une connexion WiFi vient d'être établie
- → Un nouveau SMS vient d'arriver
- → Le chargeur vient d'être branché

Intent Explicite — Navigation Interne

Un Intent explicite désigne nommément le composant cible par son nom de classe. Il est réservé à la navigation à l'intérieur de votre propre application, car vous connaissez exactement les noms des classes que vous avez créées.

4 situations typiques

Navigation entre écrans

Passer de l'accueil au détail d'un article, de la connexion au tableau de bord, du panier au paiement.

Démarrer un Service interne

Lancer votre service de téléchargement, de synchronisation ou de lecture audio en arrière-plan.

Passer des données entre écrans

Transmettre un identifiant, un nom ou un objet sérialisé d'un écran à l'autre via les Extras.

Attendre un résultat

Lancer le scanner QR code et attendre que l'utilisateur revienne avec le résultat scanné.

Exemple réel — Navigation dans Instagram

MainActivity → ReelsActivity

→ clic onglet Reels

MainActivity → CameraActivity

→ clic bouton +

ProfileActivity → EditProfileActivity

→ clic Modifier profil

FeedActivity → CommentsActivity

→ clic commentaires

InboxActivity → ChatActivity

→ clic conversation

Intent Implicite — Délégation au Système

Un Intent implicite ne désigne aucun composant précis. Il décrit uniquement l'action souhaitée et Android recherche quelle application installée peut la réaliser. Si plusieurs correspondent, une boîte de sélection est présentée à l'utilisateur.

Le processus de résolution en 4 étapes

1

L'application crée un Intent

Elle décrit l'action (ex : ACTION_SEND), le type de données (text/plain) et le contenu. Elle ne nomme aucune application cible.

2

Android consulte tous les Manifests

Le système parcourt les filtres d'Intent déclarés dans le Manifest de chaque application installée sur le téléphone.

3

Correspondance et sélection

Une seule app correspond → lancée directement. Plusieurs → boîte de dialogue. Aucune → exception ActivityNotFoundException !

4

L'utilisateur choisit

Pour le partage, l'utilisateur voit WhatsApp, Gmail, Telegram, Messenger... et sélectionne librement sa destination.

Filtres d'Intent

Un filtre d'Intent est une déclaration dans le fichier AndroidManifest.xml qui annonce au système les types d'Intents implicites que votre composant est capable de traiter. C'est ainsi qu'une application se rend visible des autres.

3 critères obligatoires pour la correspondance

① L'Action

L'Intent doit déclarer exactement la même action que le filtre.
Si votre Activity déclare ACTION_SEND, elle recevra tous les Intents de partage envoyés par d'autres apps.

② Les Données (MIME)

L'URI et le type MIME doivent correspondre.
Si le filtre déclare mimeType='image/*', votre app reçoit les partages d'images (JPEG, PNG...) mais pas les textes ni les PDF.

③ La Catégorie

CATEGORY_DEFAULT est obligatoire pour recevoir des Intents implicites via startActivity().
Sans elle, l'Activity est totalement invisible lors de la résolution.

Exemple — Comment l'app Galerie apparaît dans le menu Partager d'Instagram

1	Déclaration — à l'écriture du code, le développeur de Galerie déclare ce filtre dans le Manifest
2	Enregistrement — à l'installation de l'app, Android lit le Manifest et mémorise que Galerie sait recevoir des images
3	Résolution — Quand vous partagez depuis Instagram, Android consulte tous les filtres enregistrés et propose automatiquement Galerie dans la liste.

CHAPITRE II

Services

Exécution de tâches en arrière-plan sans interface

1. Définition et 4 caractéristiques essentielles
2. Started Service — démarré, autonome, se stoppe seul
3. Bound Service — lié, interactif via IBinder
4. Cycle de vie — les 5 callbacks à surcharger

Qu'est-ce qu'un Service ?

Un Service est un composant Android qui s'exécute en arrière-plan sans jamais afficher d'interface utilisateur. Il est conçu pour les opérations longues (téléchargement, synchronisation, lecture audio) qui ne nécessitent pas d'interaction directe.

4 caractéristiques à retenir

Aucune interface visuelle

Le Service travaille en silence. L'utilisateur le perçoit uniquement par ses effets (musique qui joue, barre de progression...).

Survit à la fermeture de l'app

Même si l'utilisateur quitte l'application, le Service continue. Essentiel pour la musique, les uploads, la navigation GPS.

Thread principal par défaut ⚠

Le Service s'exécute dans le thread UI

Exportable vers d'autres apps

Un Service peut être rendu accessible à des apps tierces via android:exported=true dans le Manifest.

Apps que vous utilisez qui utilisent des Services

🎵 Spotify

Musique continue même écran éteint. Service Started + Bound combinés.

💬 WhatsApp

Messages reçus sans ouvrir l'app. Service d'écoute permanent.

☁ Google Drive

Photos synchronisées en WiFi en arrière-plan.

🗺 Google Maps

Navigation GPS continue même si Maps n'est pas à l'écran.

🛡 Antivirus

Surveillance permanente des fichiers installés.

Started Service — Le Service Démarré

Un Started Service est déclenché par startService(). Il s'exécute de façon autonome et indépendante jusqu'à ce qu'il se stoppe lui-même (stopSelf()) ou qu'un composant l'arrête de l'extérieur (stopService()). Il ne renvoie aucun résultat.

Fonctionnement étape par étape

1 **Démarrage depuis une Activity**
L'Activity appelle startService(intent) avec les données nécessaires dans les Extras (URL, ID...).

2 **Exécution dans un thread séparé**
Le Service reçoit les données dans onStartCommand(). Il lance l'opération dans une coroutine ou un thread pour ne pas bloquer l'UI.

3 **Arrêt automatique à la fin**
Une fois terminé, le Service appelle stopSelf() pour s'arrêter proprement et libérer les ressources.

4 **Résistance aux interruptions**
START_STICKY : si Android tue le Service (manque de mémoire), il demande à être relancé automatiquement dès que possible.

3 cas d'usage courants

Téléchargement de fichier

L'utilisateur lance un téléchargement PDF. Le Service télécharge en fond et s'arrête seul quand le fichier est complet. L'Activity continue normalement.

Synchronisation de données

L'app envoie les modifications locales vers le serveur. Le Service gère la file d'attente et les reprises après erreur réseau.

Envoi d'analytics

Envoi régulier de statistiques au serveur. Le Service s'exécute brièvement et s'arrête seul après chaque envoi.

Bound Service — Le Service Lié

Un Bound Service est un Service auquel des composants peuvent se connecter (« se lier ») pour interagir avec lui via une interface IBinder. Il reste actif tant qu'au moins un client est connecté, puis s'arrête automatiquement.

Exemple Spotify — MusicService Bound + Started

Liaison

Spotify ouvert → bindService(). L'Activity contrôle la lecture via IBinder.

Dialogue

L'Activity interroge : quel titre ? Quelle progression ? Commandes : play/pause/suivant.

Découplage

Utilisateur quitte Spotify → unbindService(). La musique continue (Service aussi Started).

Reconnexion

Rouvre Spotify → bindService() à nouveau. L'Activity récupère l'état courant.

Foreground Service — Service de Premier Plan

Définition

Un Foreground Service est un Service qui effectue une opération visible par l'utilisateur via une **notification persistante obligatoire**. Il a une priorité plus haute et est rarement tué par Android.

Différence vs Started Service

Started Service : tourne en arrière-plan de façon invisible — Android peut le tuer si la mémoire manque.

Foreground Service : notif persistante visible — priorité haute — rarement tué.

⚠ **Obligation API 26+ :** startForeground() doit être appelé dans les 5 secondes suivant le démarrage, sinon Android l'arrête.

⚠ **Android 12+ :** déclaration du type dans le Manifest obligatoire (location, camera, mediaPlayback…).

Exemples réels

🎵 **Spotify / lecteur audio**
Notif avec commandes play/pause/suivant. Musique continue écran éteint.

🗺 **Google Maps / navigation GPS**
Notif de navigation active. Service GPS tourne même app en arrière-plan.

📥 **Téléchargement / upload**
Barre de progression visible. Fichier téléchargé même si l'utilisateur quitte l'app.

Cycle de Vie d'un Service — 5 Callbacks

5 méthodes à surcharger pour piloter le cycle de vie d'un Service. Android les appelle automatiquement selon les événements. Le développeur définit le comportement à chaque étape.

onCreate() *→ 1er démarrage seulement*	Appelée une seule fois à la création initiale. Initialiser ici les ressources durables : pool de threads, connexion base de données, variables d'état.
onStartCommand() *→ Chaque startService()*	Appelée à chaque startService(). Démarrer l'opération principale dans un thread séparé. La valeur de retour (START_STICKY / START_NOT_STICKY) définit le comportement si Android tue le Service.
onBind() *→ Chaque bindService()*	Appelée quand un client demande à se connecter. Retourner un IBinder qui servira d'interface de communication. Retourner null si la liaison n'est pas supportée.
onUnbind() *→ Dernier client déconnecté*	Appelée quand tous les clients ont appelé unbindService(). Libérer les ressources de communication. Retourner true pour activer onRebind() lors d'une reconnexion future.
onDestroy() *→ Avant suppression définitive*	Dernière méthode avant destruction. Libérer TOUTES les ressources : threads, listeners, connexions réseau. Négliger cela = fuites mémoire assurées.

CHAPITRE III

BroadcastReceivers

Écouter et réagir aux événements système

1. Définition — principe de fonctionnement
2. 6 exemples réels dans les apps du quotidien
3. Enregistrement statique vs dynamique
4. Diffusion normale vs diffusion ordonnée

BroadcastReceiver — Définition

Un BroadcastReceiver est un composant Android qui écoute des Intents diffusés à l'échelle du système entier. Il réagit aux événements sans interface utilisateur, en exécutant rapidement du code dans sa méthode onReceive().

Le mécanisme en 4 étapes

1

Un événement se produit

Batterie faible, SMS reçu, WiFi connecté, démarrage du système, casque branché, chargeur branché... Tout événement peut déclencher une diffusion.

2

Android diffuse un Intent

Le système crée un Intent décrivant l'événement et appelle sendBroadcast(). L'Intent est envoyé à tous les composants enregistrés pour ce type d'événement.

3

Les Receivers notifiés

Android parcourt la liste des Receivers enregistrés. Ceux dont le filtre correspond reçoivent l'Intent et leur méthode onReceive() est appelée.

4

Action rapide dans onReceive()

Le Receiver exécute une action courte : afficher une notification, écrire en BDD, ou démarrer un Service pour un traitement plus long.

Enregistrement Statique vs Dynamique

Il existe deux façons d'enregistrer un BroadcastReceiver. Le choix dépend du besoin : écouter en permanence (même app fermée) ou seulement pendant la durée de vie d'un composant.

Statique — dans AndroidManifest.xml

Déclaré directement dans le fichier Manifest.

Actif dès l'installation, même si l'app n'a jamais été lancée.

Android le démarre automatiquement dès que l'événement correspondant se produit.

Idéal pour :

→ Démarrage du système (BOOT_COMPLETED)

→ Réception SMS (SMS_RECEIVED)

→ Événements batterie (BATTERY_LOW)

→ Installation de packages

Dynamique — dans le Code Kotlin/Java

Enregistré dans le code d'un composant (Activity, Service) via registerReceiver().

Actif seulement pendant la vie du composant. Appel obligatoire à unregisterReceiver() dans onStop() ou onDestroy() pour éviter les fuites mémoire.

Idéal pour :

→ Changements réseau (CONNECTIVITY_CHANGE)

→ Mise à jour UI en temps réel

→ Événements liés à l'écran visible

→ Tout événement lié à une Activity ouverte

CHAPITRE IV

ContentProviders

Partage structuré et sécurisé de données

1. Définition — rôle et architecture
2. L'URI de contenu — adresse unique d'une ressource
3. Le ContentResolver — l'intermédiaire obligatoire
4. Opérations CRUD : query, insert, update, delete

ContentProvider — Définition & Architecture

Un ContentProvider est une couche d'abstraction qui permet à une application d'exposer ses données à d'autres applications de façon sécurisée. Les données sont présentées sous forme de tables accessibles via des URI standardisées.

Pourquoi utiliser un ContentProvider ?

Partage inter-applications

Sans ContentProvider, les données d'une app sont inaccessibles aux autres. Chaque app vit dans un sandbox isolé. Le Provider crée un pont contrôlé.

Interface d'accès uniforme

Peu importe si les données viennent de SQLite, d'un fichier ou du réseau : l'interface est toujours la même (query, insert, update, delete).

Contrôle fin des permissions

readPermission et writePermission séparées. Une app peut avoir le droit de lire sans pouvoir écrire — contrôle précis.

Notifications de changements

Les composants s'abonnent via ContentObserver pour être notifiés automatiquement quand les données sont modifiées.

ContentProviders intégrés dans Android

ContactsContract

Contacts, numéros, emails — Permission : READ/WRITE_CONTACTS

MediaStore

Photos, vidéos, fichiers audio — Permission : READ_EXTERNAL_STORAGE

CalendarContract

Événements, rappels — Permission : READ/WRITE_CALENDAR

CallLog

Historique des appels — Permission : READ_CALL_LOG

SMS / MMS

Messages texte et multimédias — Permission : READ_SMS

URI de Contenu & ContentResolver

Pour accéder à un ContentProvider, il faut deux outils : une URI qui identifie précisément la ressource demandée, et un ContentResolver qui sert d'intermédiaire obligatoire entre l'application et le Provider.

Anatomie d'une URI de contenu

```
content  ://  com.android.contacts  /  contacts  /  42
```

content://

Schéma fixe. Indique que la ressource est gérée par un ContentProvider (comme http:// pour le web).

com.android.contacts

Authority : identifiant unique du ContentProvider cible. Android l'utilise pour trouver le bon Provider.

/contacts

Path : indique la table ou type de données. Peut avoir plusieurs niveaux (/contacts/phones).

/42

Identifiant optionnel de l'enregistrement précis. Sans lui, la requête porte sur toute la table.

Le ContentResolver — chef d'orchestre des accès

Intermédiaire obligatoire — Votre app n'accède jamais directement au ContentProvider. Elle passe toujours par le ContentResolver : getContentResolver().query(uri, ...)

Résolution automatique — Le ContentResolver lit l'authority de l'URI et trouve automatiquement le bon Provider, qu'il soit local ou dans une app tierce.

Services supplémentaires — Gère les ContentObserver (notifications temps réel), les opérations en batch, et vérifie les permissions avant chaque accès.

CHAPITRE V

Permissions

Protéger les ressources et la vie privée des utilisateurs

1. Pourquoi le modèle de permissions existe-t-il ?
2. Permissions Normales vs Permissions Dangereuses
3. Permissions par composant Android
4. Runtime Permissions — demander dynamiquement

Architecture des Permissions Android

Le modèle de permissions Android protège l'utilisateur en contrôlant précisément ce que chaque application peut faire. Sans permission déclarée, une app ne peut ni lire les contacts, ni accéder à la caméra, ni utiliser le GPS.

2 niveaux de permissions

Permissions Normales — accordées automatiquement

Risque faible pour la vie privée. Android les accorde sans dialog, simplement déclarées dans le Manifest.

Exemples :

→ INTERNET — accéder au réseau
→ VIBRATE — faire vibrer l'appareil
→ RECEIVE_BOOT_COMPLETED — démarrer après reboot
→ ACCESS_NETWORK_STATE — vérifier la connexion
→ NFC · SET_ALARM · CHANGE_WIFI_STATE

Permissions Dangereuses — demandées à l'utilisateur

Accès à des données sensibles ou des ressources critiques. Depuis Android 6.0, l'utilisateur doit les accorder explicitement.

Exemples :

→ READ_CONTACTS / WRITE_CONTACTS
→ ACCESS_FINE_LOCATION (GPS précis)
→ CAMERA
→ RECORD_AUDIO
→ READ_SMS · READ_CALL_LOG · READ_CALENDAR

Runtime Permissions — Demander à l'Exécution

Depuis Android 6.0 (API 23), les permissions dangereuses se demandent dynamiquement au moment précis de leur utilisation. Le développeur doit vérifier, demander, et traiter le résultat — en gérant aussi le cas du refus.

Le processus en 4 étapes — Exemple : accéder à la caméra

1 Déclarer dans AndroidManifest.xml

Toute permission, même runtime, doit d'abord être déclarée dans le Manifest. Sans déclaration, Android refusera la demande même si l'utilisateur accepte.

```
<uses-permission
 android:name=

"android.permission
 .CAMERA" />
```

2 Vérifier si déjà accordée

Avant d'afficher une dialog, vérifier avec checkSelfPermission(). Si PERMISSION_GRANTED → lancer directement l'action sans déranger l'utilisateur.

```
if(checkSelf
Permission(CAME
RA)
 == GRANTED)
  openCamera()
```

3 Demander la permission

Si non accordée → appeler requestPermissions() avec un code de requête. Android affiche automatiquement la boîte de dialogue officielle.

```
requestPermissi
ons(

arrayOf(CAMERA)
,
 REQ_CAMERA)
```

4 Traiter le résultat

Le callback onRequestPermissionsResult() reçoit la réponse. Vérifier le code et la réponse. Toujours gérer le refus : expliquer pourquoi c'est nécessaire.

```
if(grantResults
[0]
 == GRANTED)
  openCamera()
else showWhy()
```